PROVENDA - GESTION COMPLÈTE

MANUEL DE RÉFÉRENCE FONCTIONNEL & TECHNIQUE EXHAUSTIF

Documentation intégrale de toutes les fonctions, modules et règles métier

Application :	Provenda - Solution de Gestion Opérationnelle & Financière
Type de Système :	ERP Métier Provenderie & Point de Vente (POS) Temps Réel
Monnaie de Référence :	Franc CFA (FCFA / XAF)
Méthode de Valorisation :	Coût Moyen Pondéré (CMP) avec intégration des frais d'approche
Architecture & Framework :	React 19 + TypeScript + Tailwind CSS (Vite Engine)
Couverture :	100% des Fonctions, Formules de calcul, Modèles & Sécurité RBAC
Date d'Édition :	25 septembre 2026
Statut du Document :	Officiel - Guide Administrateur & Développeur

RÉSUMÉ EXÉCUTIF & STRUCTURE DU DOCUMENT

Ce document recense de manière exhaustive l'ensemble des fonctionnalités et fonctions logicielles qui constituent l'application Provenda. Chaque fonction du gestionnaire d'état central (AppContext), chaque formule mathématique (CMP, Marges, Rapprochement d'inventaire, Plafonds de remise), chaque composant d'interface utilisateur (POS, Stocks, Achats, Facturation, Clients, Fournisseurs, Rapports, Piste d'audit, Paramètres) et chaque modalité de sécurité (RBAC) y sont scrupuleusement décrits avec leurs règles de validation, entrées/sorties et cas d'usage.

• Chapitre 1 : Architecture Globale, Modèles de Données & RBAC (Rôles & Permissions)
• Chapitre 2 : Moteur de Stock, Conversion d'Unités & Calcul Dynamique du CMP
• Chapitre 3 : Terminal Point de Vente (POS), Encaissement Multi-Moyens & Remises
• Chapitre 4 : Gestion des Achats, Réceptions Fournisseurs & Frais d'Approche
• Chapitre 5 : Facturation, Crédits Clients, Échéances & Gestion des Avoirs
• Chapitre 6 : Sessions d'Inventaire Physique & Régularisations de Stock
• Chapitre 7 : Rapports Financiers, Marges Commerciales & Journaux d'Audit
• Chapitre 8 : Répertoire Exhaustif des Fonctions Code (Nom, Paramètres, Fonctionnement)

Provenda © 2026 - Système Intégré de Gestion de Provenderie

1. Architecture Globale & Matrice de Sécurité (RBAC)

Provenda est architecturé autour d'un gestionnaire d'état centralisé React Context (AppContext), garantissant la cohérence transactionnelle entre les stocks, les caisses, les achats, les créances clients et la piste d'audit. La sécurité repose sur un contrôle d'accès basé sur les rôles (Role-Based Access Control - RBAC).

1.1 Matrice des Rôles & Droits d'Accès Applicatifs

Rôle	Plafond Remise	Point de Vente (POS)	Gestion Stocks & CMP	Achats & BL	Clients & Dettes	Paramètres & Sauvegardes
Administrateur	100% max	Accès Total	Création, CMP, Ajust.	Création, Validation	Plafonds, Règl., Avoirs	Accès Complet & Multi-Magasins
Responsable	15% max	Accès Total	Consultation, Ajust.	Brouillons, Validation	Règlement, Dérogation	Consultation & Rapports
Caissier	0% (Interdit)	Vente & Encaissement	Consultation seule	Aucun accès	Encaissement dettes	Aucun accès
Magasinier	0% (Interdit)	Aucun accès	Ajust., Mouvements, Inv.	Réception physique BL	Aucun accès	Aucun accès

1.2 Architecture Multi-Magasins (Points de Vente)

L'application permet d'administrer plusieurs points de vente physiques (ex: Boutique Principale, Dépôt Port, Annexe Marché). Le sélecteur de magasin actif filtre dynamiquement les ventes, inventaires et réceptions. Chaque utilisateur peut être rattaché à un point de vente spécifique, et l'Administrateur peut basculer d'un magasin à l'autre en un clic depuis l'en-tête.

1.3 Entités & Modèles de Données Principaux

Entité	Clés & Identifiants	Champs Critiques	Rôle Métier
Product	id, code (ex: PRD-001)	baseUnit, saleUnit, conversionFactor, cmp, stockCurrentBaseUnit	Article provenderie géré au kg ou sac avec son CMP.
Invoice	id, number (FAC-AAAA-XXXXXX)	customerId, items[], totalNet, totalCostOfGoodsSold, grossMargin	Facture de vente avec traçabilité marge & TVA.
Purchase	id, referenceNumber (ACH-...)	supplierId, items[], transportFees, otherFees, status	Bon d'approvisionnement intégrant les frais d'approche.
StockMovement	id, timestamp	productId, type, quantityBaseUnit, costUnitCMP, justification	Journal inaltérable de tous les flux physiques d'entrée/sortie.
InventorySession	id, referenceNumber (INV-...)	items[], theoreticalStock, physicalStock, varianceValue	Session de comptage physique avec calcul des écarts valorisés.
Customer	id, code (CLI-001)	creditLimit, currentDebt, type (éleveur, coopérative, revendeur)	Fiche tiers client et contrôle strict des plafonds de crédit.
AuditLog	id, timestamp	userId, action, targetObject, oldValue, newValue, justification	Journal de conformité et de sécurité ISO pour audits financiers.

2. Moteur de Stock & Algorithme du Coût Moyen Pondéré (CMP)

2.1 Formule Mathématique Officielle du CMP

FORMULE DU COÛT MOYEN PONDÉRÉ APRÈS RÉCEPTION :

```
Nouveau CMP = ( (Stock_Init × Ancien_CMP) + Coût_Total_Entrée ) / ( Stock_Init + Qté_Entrée )
```

Où Coût_Total_Entrée = (Qté_Entrée × Prix_Achat_Unitaire) + Frais_Transport + Autres_Frais_Approche

2.2 Règle d'Incorporation des Frais d'Approche

Conformément aux normes comptables SYSCOHADA et IFRS pour les provenderies d'élevage, le coût d'acquisition d'une matière première (Maïs, Tourteau de soja, Concentré chair) inclut non seulement le prix facturé par le fournisseur, mais également les frais de transport, déchargement et manutention. Dans Provenda, l'option 'Frais de transport incorporés au CMP' ventile automatiquement ces surcoûts au prorata des quantités reçues, évitant ainsi toute sous-évaluation de la valeur de stock et tout calcul erroné des marges.

2.3 Conversion d'Unités (Unité de Base vs Unité Commerciale)

Conditionnement	Unité Commerciale	Unité de Base Stock	Facteur de Conversion	Exemple Pratique
Sac de 50 kg (Aliment Chair/Pondeuse)	sac	kilogramme (kg)	50	Vente de 2 sacs = Déduction automatique de 100 kg en stock
Sac de 25 kg (Aliment Démarrage)	sac	kilogramme (kg)	25	Vente de 4 sacs = Déduction automatique de 100 kg en stock
Vente au détail au kilo (Vrac)	kilogramme	kilogramme (kg)	1	Vente de 12.5 kg = Déduction exacte de 12.5 kg en stock
Bidon de 20 Litres (Huile/Mélasse)	bidon	litre (L)	20	Réception de 5 bidons = Augmentation de 100 L en stock

3. Terminal Point de Vente (POS) & Encaissement

Le terminal de caisse POS est conçu pour un enregistrement rapide des commandes au comptoir, avec recherche instantanée par code-barres ou nom, gestion des conditionnements (sac/kg), calcul automatique des marges en arrière-plan, vérification stricte des remises autorisées selon le rôle et validation des encaissements multi-moyens (Espèces, Mobile Money Orange/MTN, Virement ou Crédit Client).

3.1 Calculateur de Rentabilité & Marge Brute en Temps Réel

```
1. Total Brut = Somme ( Quantité × Prix Unitaire Catalogue )
2. Chiffre d'Affaires Net = Total Brut - Remise Globale Accordée
3. Coût des Ventes (CMV) = Somme ( Quantité Base × CMP du Produit )
4. Marge Brute = CA Net - CMV  |  Taux de Marge (%) = (Marge / CA Net) × 100
```

3.2 Règles d'Attribution des Remises Commerciales

Profil Utilisateur	Remise Autorisée	Action si Dépassement	Traçabilité Requise
Caissier	0% (Aucune remise)	Bloqué par le système. Nécessite accord responsable.	Impossible d'enregistrer sans dérogation
Responsable	Jusqu'à 15% maximum	Au-delà de 15% : bloqué, nécessite administrateur.	Nom, motif et taux archivés sur la facture
Administrateur	Jusqu'à 100% (Dérogation totale)	Libre avec obligation de saisir un motif.	Enregistré dans le journal d'audit avec justification

8. Répertoire Exhaustif des Fonctions du Code Source (AppContext)

Ce chapitre détaille chaque fonction implémentée dans le gestionnaire central AppContext.tsx. Il spécifie pour chaque méthode : sa signature TypeScript complète, ses données d'entrée, ses valeurs de retour, et l'algorithme métier exécuté de bout en bout.

Thème, Internationalisation & Navigation

Fonction	Signature TypeScript	Description Opérationnelle & Algorithme
toggleTheme()	() => void	Bascule entre le mode sombre (dark) et le mode clair (light). Sauvegarde le choix dans localStorage sous provenda_theme.
setLanguage(lang)	(lang: "fr" | "en") => void	Définit la langue globale de l'interface. Sauvegarde la sélection dans localStorage et adapte tous les libellés via i18n.
t(key)	(key: string) => string	Résout une clé de traduction hiérarchique (ex: "nav.pos", "stock.cmp") selon la langue active courante avec fallback.

Gestion Multi-Magasins (Points de Vente)

Fonction	Signature TypeScript	Description Opérationnelle & Algorithme
setCurrentSalesPointId(id)	(id: string) => void	Active un point de vente de travail. Filtre instantanément les flux de ventes, commandes et inventaires.
addSalesPoint(sp)	(sp: Omit<SalesPoint, "id"|"createdAt">) => void	Crée une nouvelle succursale avec son code POS unique, adresse et responsable. Consigne dans l'audit.
updateSalesPoint(sp)	(sp: SalesPoint) => void	Met à jour les coordonnées, téléphone et gérant d'un point de vente existant avec traçabilité d'audit.
deleteSalesPoint(id)	(id: string) => void	Supprime un point de vente secondaire. Le magasin principal ne peut être supprimé.

Authentification & Rôles Utilisateurs

Fonction	Signature TypeScript	Description Opérationnelle & Algorithme
login(email, password)	(email, password) => { success, error }	Authentifie un utilisateur par email et mot de passe. Vérifie que le compte est actif et met à jour lastLogin.
logout()	() => void	Déconnecte la session utilisateur active, nettoie les données d'accès et redirige vers la page d'accès.
switchRole(role)	(role: Role) => void	Permet de basculer instantanément de casquette (admin, responsable, caissier, magasinier) pour vérifier les permissions.
addUser(user)	(user) => { success, error }	Crée un nouvel utilisateur avec attribution de son rôle, son plafond de remise et son magasin assigné.
updateUser(user)	(user: User) => void	Met à jour les informations, mot de passe, rôle ou statut actif/inactif d'un employé avec traçabilité complète.
deactivateUser(id)	(id: string) => void	Désactive le compte d'un employé empêchant toute connexion future sans effacer son historique comptable.

Achats, Réceptions & Recalcul du CMP

Fonction	Signature TypeScript	Description Opérationnelle & Algorithme
createPurchaseDraft(p, spId)	(purchase, targetSalesPointId?) => string	Génère un bon de commande au statut "brouillon" avec référence ACH-AAAA-XXXXXX, articles et frais d'approche.
validatePurchaseReception(id)	(purchaseId: string) => void	Cœur du moteur CMP : calcule le nouveau CMP unitaire de chaque produit reçu, incrémente le stock et clôture le bon.
cancelPurchase(id, reason)	(purchaseId: string, reason: string) => void	Passe le bon d'achat au statut "annulé" avec motif obligatoire. Ne modifie pas les stocks physiques.

Ventes, Caisse POS, Remises & Facturation

Fonction	Signature TypeScript	Description Opérationnelle & Algorithme
createSale(...)	(customerId, items, discount?, payment?, notes?, isCredit?, creditDueDate?, spId?) => { success, invoiceId, error }	Exécute la transaction de vente intégrale : vérifie le stock disponible, contrôle les plafonds de remise, valide le crédit client, calcule la marge brute, génère la facture FAC, décrémente les stocks, enregistre le paiement et déclenche l'audit.
cancelInvoiceWithCreditNote(...)	(invoiceId: string, reason: string) => { success, error }	Annule une facture de vente : génère un Avoir officiel AVO-AAAA-XXXXXX, réintègre les stocks, annule la créance client et archive le motif.

Fonction	Signature TypeScript	Description Opérationnelle & Algorithme
recordDebtPayment(...)	(invoiceId, amount, method, ref?, notes?) => { success, error }	Enregistre le versement d'un client sur une facture à crédit. Réduit le solde de la facture, diminue la dette du client et bascule le statut si soldé.

Ajustements de Stock & Inventaires

Fonction	Signature TypeScript	Description Opérationnelle & Algorithme
recordStockAdjustment(...)	(productId, qtyChangeBaseUnit, type, justification) => { success, error }	Effectue une régularisation manuelle de stock (perte, casse, consommation interne, correction). Recalcule le stock, génère le mouvement et l'audit.
createInventorySession(cat?)	(categoryFilter?: string) => string	Crée une session d'inventaire physique avec capture instantanée du stock théorique de tous les produits.
validateInventorySession(...)	(sessionId, countedItems, notes?) => { success, error }	Confronte le stock physique au stock théorique, calcule les écarts valorisés au CMP, ajuste les stocks réels et clôture la session.

Sauvegarde, Restauration & Sécurité Système

Fonction	Signature TypeScript	Description Opérationnelle & Algorithme
exportSystemBackup()	() => string	Exporte l'intégralité des tables de données sous forme de chaîne JSON horodatée téléchargeable.
restoreSystemBackup(json)	(jsonString: string) => { success, message }	Valide la cohérence du fichier JSON, restaure l'état complet de la base et consigne dans l'audit.
resetToDefaultData()	() => void	Réinitialise l'application aux données de démonstration d'usine pré-calibrées.
getLowStockProducts()	() => Product[]	Sélecteur retournant tous les articles actifs dont le stock actuel est inférieur ou égal au seuil minimum d'alerte.
getOutOfStockProducts()	() => Product[]	Sélecteur retournant tous les articles actifs dont le stock est strictement épuisé (<= 0).